

Project Title: Securing a Basic Website

Date: 15th March 2025

Prepared by: Dipu Kumar

Website Link : <http://localhost/simple-php-website/contact.php>

1. Original Website Description

The initial website was a simple PHP project with the following pages:

- **index.html**: A static homepage.
- **contact.php**: A form to collect user data (name and email).
- **thankyou.php**: A page displaying the submitted data.

Vulnerabilities Identified:

1. No **input validation**, which made the website susceptible to SQL injection and other malicious inputs.
2. No **HTTPS**, leaving data transmission insecure.
3. Lack of **XSS prevention**, allowing potential cross-site scripting attacks.

2. Security Improvements Made:

2.1 Input Validation

- **Issue Identified**: User inputs were directly processed without sanitization, leaving the site open to malicious inputs (e.g., SQL injection).
- **Solution**:

- Added validation and sanitization using PHP functions like `filter_input()` and `htmlspecialchars()`.

- Updated `thankyou.php`:

```
<?php
```

```
➡ if ($_SERVER["REQUEST_METHOD"] == "POST") {  
    $name = filter_input(INPUT_POST, 'name', FILTER_SANITIZE_STRING);  
    $email = filter_input(INPUT_POST, 'email', FILTER_VALIDATE_EMAIL)  
    if (!$email) {
```

```

        die("<p>Invalid email format!</p>" }

echo "<h1>Thank You, " . htmlspecialchars($name, ENT_QUOTES, 'UTF-8') .
"!</h1>";

echo "<p>We will reach out to you at " . htmlspecialchars($email,
ENT_QUOTES, 'UTF-8') . " soon.</p>";

}

?>

```

2.2 HTTPS Enforcement

- **Issue Identified:** The website used HTTP, making data vulnerable to interception.
- **Solution:**
 - Implemented HTTPS by generating a **self-signed certificate**.
 - Configured Apache to enforce HTTPS:

- Created and installed an SSL certificate using OpenSSL:

```

➡ sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
    -keyout /etc/ssl/private/localhost-selfsigned.key \
    -out /etc/ssl/certs/localhost-selfsigned.crt

```

Updated the Apache configuration file (/etc/apache2/sites-available/default-ssl.conf) with the certificate paths:

```

➡ SSLEngine on

SSLCertificateFile /etc/ssl/certs/localhost-selfsigned.crt
SSLCertificateKeyFile /etc/ssl/private/localhost-selfsigned.key

```

Redirected HTTP to HTTPS in the .htaccess file:

```

➡ RewriteEngine On

RewriteCond %{HTTPS} off

RewriteRule ^(.*)$ https://%{HTTP_HOST}%{REQUEST_URI} [L,R=301]

```

2.3 XSS Prevention

- **Issue Identified:** Data output was not escaped, allowing the execution of malicious scripts.

- **Solution:**

- Escaped all user-provided data using htmlspecialchars() in thankyou.php:

 echo "<h1>Thank You, " . htmlspecialchars(\$name, ENT_QUOTES, 'UTF-8') .
"!</h1>";

echo "<p>We will reach out to you at " . htmlspecialchars(\$email,
ENT_QUOTES, 'UTF-8') . " soon.</p>";

3. Challenges and Solutions

1. **Challenge:** Permission Denied Errors **Solution:** Used sudo to modify files in /var/www/html.
2. **Challenge:** DNS Issues with Certbot **Solution:** Switched to a self-signed certificate for local development.
3. **Challenge:** Testing Input Validation **Solution:** Tested various input cases (e.g., invalid emails, special characters) to ensure data sanitization was effective.

4. Final Outcome

The website is now more secure with the following updates:

1. Input validation ensures only sanitized and validated data is processed.
2. HTTPS encrypts communication, protecting user data during transmission.
3. XSS prevention ensures that malicious scripts cannot be executed.

These changes align with best practices for securing PHP-based websites and significantly reduce vulnerabilities.